MÁSTER EN CIBERSEGURIDAD

MÓDULO 11. TRABAJO FIN DE MÁSTER (TFM)

La Crisis de la Gestión de Secretos: Secret Sprawl

Andrea Osma Rafael

Campus de Ciberseguridad

Resumen TFM del Máster de Ciberseguridad de la ENIIT sobre la problemática del Secret Sprawl en entornos DevSecOps. Se analiza la fuga de credenciales en repositorios y pipelines de CI/CD, proponiendo una arquitectura basada en bóvedas criptográficas y gestión dinámica de identidades.

Keywords: Ciberseguridad · DevSecOps · Secret Sprawl · Gestión de Secretos · Zero Trust

Índice general

MÁSTER EN CIBERSEGURIDAD MÓDULO 11. TRABAJO FIN DE MÁSTER (TFM) . . . . . .
Andrea Osma Rafael
1. Estado del Arte 2
1.1. Introducción: El Colapso del Perímetro Tradicional 2
1.1.1. El Paradigma Zero Trust y el Cumplimiento Normativo 2
1.2. La Crisis de la Gestión de Secretos: Secret Sprawl 2
1.2.1. Taxonomía de la Dispersión de Secretos 2
1.2.2. El Desafío de la Escala: Máquinas vs. Humanos 3
1.2.3. Riesgos de Seguridad Derivados del Secret Sprawl 3
1.2.4. El Ciclo de Vida del Secreto 4
1.2.5. Integración Transversal en el SDLC (DevSecOps) 4
1.3. Trazabilidad Forense y Dispositivos de Auditoría 5
2. Motivación 5
3. Hipótesis 6
4. Tesis: Arquitectura DevSecOps para la Gestión Dinámica de Secretos 6
4.1. Diseño de la Arquitectura Lógica 7
4.2. Resolución del Problema del "Secreto Cero" (Secret Zero) 7
4.3. Despliegue Inmutable y Operativa de Ingeniería 7
5. Demostración 8
5.1. Entorno de Pruebas 8
5.2. Estrategia de Despliegue mediante IaC 10
5.2.1. Instalación de dependencias en Proxmox 10
5.2.2. Aprovisionamiento Declarativo con Terraform 11
6. Conclusiones 16
7. Glosario 17
8. Bibliografía 19

1. Estado del Arte

1.1. Introducción: El Colapso del Perímetro Tradicional

La evolución de la infraestructura IT, desde los centros de datos locales hacia la computación en la nube y las arquitecturas de microservicios, ha forzado una reevaluación total de los modelos de confianza. El modelo de seguridad perimetral, basado en la premisa de una red interna segura protegida por firewalls (Castillo y Foso), ha demostrado ser ineficaz en entornos donde las cargas de trabajo son efímeras y se distribuyen en múltiples nubes.

1.1.1. El Paradigma Zero Trust y el Cumplimiento Normativo Según la publicación especial NIST SP 800-207, la Arquitectura Zero Trust (ZTA) es un modelo de ciberseguridad que asume que no existe confianza implícita concedida a los activos o cuentas de usuario basándose únicamente en su ubicación física o de red. En lugar de defender un perímetro estático, la autenticación y autorización se evalúan de forma dinámica y continua por cada solicitud de acceso, protegiendo los recursos de manera individualizada.

A nivel regulatorio, este cambio de paradigma se ha materializado en marcos europeos recientes como la directiva NIS2 y DORA (Digital Operational Resilience Act). Estas normativas exigen a las organizaciones, especialmente en infraestructuras críticas y el sector financiero, implementar controles estrictos y demostrables sobre la gestión de identidades, la resiliencia operativa y el riesgo de la cadena de suministro. La rotación automatizada de credenciales y la auditoría continua pasan de ser buenas prácticas a ser obligaciones legales estrictas.

En el paradigma actual, la red es dinámica y, por definición, hostil. Por tanto, la seguridad debe desplazarse desde la red hacia la identidad. Este cambio sitúa a la Gestión de Identidad y Acceso (IAM) no como un servicio de soporte, sino como el plano de control central de la seguridad de la información. Este TFM analiza soluciones diseñadas para mitigar los riesgos derivados de la proliferación de identidades no humanas.

1.2. La Crisis de la Gestión de Secretos: Secret Sprawl

1.2.1. Taxonomía de la Dispersión de Secretos El término secret sprawl define la distribución descontrolada de credenciales (claves API, certificados, tokens, contraseñas) a lo largo del ciclo de vida del desarrollo de software (SDLC). En un entorno de Infraestructura como Código (IaC), el riesgo aumenta considerablemente.

- **Fugas en Control de Versiones:** La inmutabilidad de los sistemas como Git implica que un secreto subido accidentalmente persiste en el historial, requiriendo un proceso complejo de rewriting history o la invalidación inmediata de la credencial.

- **Variables de Entorno Inseguras:** El uso de archivos .env o variables de sistema sin cifrar en servidores de CI/CD (Jenkins, GitHub Actions) expone credenciales a cualquier proceso con privilegios de lectura en el entorno.
- **Certificados X.509 y Claves SSH Compartidas:** La emisión de certificados de larga duración o el uso de pares de claves SSH estáticas sin el respaldo de una Autoridad de Certificación (CA) efímera provoca que, si el material privado es comprometido, el acceso se mantenga de forma prolongada sin ser detectado.
- **Credenciales Hardcodeadas en Código Fuente:** La práctica de incrustar tokens o contraseñas directamente en los repositorios expone la información a cualquier usuario con privilegios de lectura y propaga el secreto a través de los artefactos de construcción.

1.2.2. El Desafío de la Escala: Máquinas vs. Humanos Investigaciones recientes sugieren que las identidades de máquinas (servicios, contenedores, bots) crecen a un ritmo cinco veces superior al de las identidades humanas. A diferencia de un operador humano, que puede validar su identidad aportando biometría o un token de hardware en un flujo MFA† tradicional, una carga de trabajo automatizada carece de este factor interactivo ("algo que eres" o "algo que tienes").

Esto exige sustituir el MFA† humano por la validación concurrente de múltiples atributos criptográficos y contextuales. Se requiere la presentación de credenciales como certificados TLS† mutuos (mTLS†) o tokens JWT firmados, combinados con la verificación de la IP de origen, el namespace o la rama del repositorio en tiempo de ejecución. Se espera que la validación estricta de estos metadatos limite la emisión de secretos exclusivamente a las cargas de trabajo legítimas.

1.2.3. Riesgos de Seguridad Derivados del Secret Sprawl El almacenamiento incontrolado y la exposición de credenciales generan vectores de ataque críticos en las infraestructuras modernas. La literatura académica señala que la exposición de secretos en pipelines de CI/CD y en repositorios de código facilita la ejecución de los siguientes escenarios de compromiso:

- **Movimiento Lateral y Escalada de Privilegios:** En arquitecturas planas o basadas en microservicios, un atacante que obtiene un token de acceso básico puede pivotar hacia sistemas críticos o bases de datos si no existe una autenticación estricta.
- **Ataques a la Cadena de Suministro (Supply Chain Attacks):** La inyección de código malicioso en los flujos de integración continua a menudo se logra explotando credenciales expuestas en los repositorios, lo que compromete las fases de construcción y despliegue del software de forma automatizada.
- **Persistencia a Largo Plazo:** Las credenciales estáticas que carecen de rotación permiten a los atacantes mantener un acceso indetectable a la infraestructura (comportamiento esperado tras una intrusión exitosa), incluso después de que los equipos de respuesta a incidentes mitiguen la vulnerabilidad técnica inicial que permitió la entrada.

1.2.4. El Ciclo de Vida del Secreto Para mitigar los riesgos asociados a la dispersión de credenciales, la gestión de secretos debe tratarse como un proceso continuo e integrado. Los secretos se consideran artefactos de primer nivel que requieren un ciclo de vida definido desde su creación hasta su destrucción. Este ciclo consta de las siguientes fases:

1. **Generación:** Creación de la credencial, preferiblemente de forma dinámica y asociando un Tiempo de Vida (TTL) estrictamente limitado a la misma.
2. **Almacenamiento Seguro:** Custodia del secreto en un almacén centralizado que proporcione cifrado y aplique controles de acceso basados en políticas (Policy-as-Code†) para definir quién, y bajo qué condiciones, puede interactuar con el dato.
3. **Distribución e Inyección:** Entrega temporal del secreto a la identidad que lo solicita (humana o de máquina) en tiempo de ejecución, evitando en todo momento su escritura en disco.
4. **Rotación:** Renovación periódica o basada en eventos de las credenciales, un proceso fundamental para invalidar versiones anteriores y acotar drásticamente la ventana de exposición ante una posible brecha.
5. **Revocación y Auditoría:** Invalidación del secreto en el sistema de destino tras expirar su TTL, manteniendo un registro inmutable de todas las operaciones para facilitar labores de cumplimiento normativo y diagnóstico forense.

1.2.5. Integración Transversal en el SDLC (DevSecOps) La gestión de identidades y secretos interactúa a lo largo de todo el Ciclo de Vida del Desarrollo de Software (SDLC). Aplicar el principio de shift-left security exige implementar controles técnicos desde las fases más tempranas:

- **Fase de Código (Code and Commit):** Constituye la primera línea de defensa. Dado que es habitual que se introduzcan credenciales hardcodeadas de manera inadvertida, en esta etapa se despliegan escaneos de secretos y hooks de pre-commit en los repositorios locales para bloquear la inyección en el control de versiones.
- **Fase de Construcción (Build and CI):** Los servidores de Integración Continua requieren acceso a repositorios de artefactos y entornos de prueba. El uso de variables de entorno estáticas en esta fase representa un riesgo significativo de fuga de datos. Los secretos deben ser inyectados dinámicamente y enmascarados para evitar su volcado accidental en los registros (logs) de ejecución.
- **Fase de Pruebas (Test):** Las herramientas de análisis dinámico e interactivo (DAST† e IAST†) necesitan credenciales temporales para ejecutar sus escaneos en los entornos de validación. Esta fase garantiza la separación estricta entre las credenciales de prueba y las de producción.
- **Fase de Despliegue y Operación (Deploy and Run):** En la etapa final, las cargas de trabajo contenerizadas u orquestadas solicitan sus credenciales en tiempo de ejecución. El gestor central valida la identidad criptográfica de la carga de trabajo y entrega un acceso efímero vinculado a políticas contextuales.

1.3. Trazabilidad Forense y Dispositivos de Auditoría

Un pilar fundamental de la Arquitectura Zero Trust es la visibilidad total sobre las peticiones de acceso. Durante las pruebas, se habilitó el motor de auditoría de Vault[†] para registrar cada interacción en formato JSON. Este registro inmutable captura el identificador de la entidad solicitante, la política evaluada, la ruta del secreto y la marca de tiempo exacta.

La exportación de estos registros estructurados hacia un sistema SIEM centralizado permite la detección de anomalías en tiempo real, como múltiples intentos de acceso denegados desde una IP específica, garantizando así los requisitos de resiliencia y auditoría continua exigidos por marcos normativos actuales.

2. Motivación

La transición de arquitecturas monolíticas a ecosistemas distribuidos, contenerizados y orientados a microservicios ha redefinido radicalmente el perímetro de seguridad clásico. En este paradigma actual, la red subyacente se asume como un entorno intrínsecamente hostil, desplazando a la identidad hacia el centro del plano de control. Este avance arquitectónico, si bien ha dotado de agilidad al desarrollo, ha precipitado una crisis operativa sin precedentes: la proliferación descontrolada de identidades no humanas, tales como servicios, contenedores y automatizaciones de despliegue. A diferencia de los usuarios humanos, estas entidades requieren mecanismos de autenticación criptográfica automatizados y de alta frecuencia, lo que ha derivado en el fenómeno conocido como Secret Sprawl (dispersión de secretos) a lo largo del ciclo de vida del desarrollo de software (SDLC).

En paralelo, la consolidación de las metodologías ágiles ha convertido a los repositorios de código y a las plataformas de Integración y Despliegue Continuo (CI/CD) en la columna vertebral de la ingeniería de software y, consecuentemente, en vectores de ataque de alto valor. La persistencia técnica de incrustar credenciales estáticas (hardcoding) o de utilizar variables de entorno debilmente protegidas expone a las infraestructuras a riesgos severos, facilitando el movimiento lateral y el compromiso de la cadena de suministro. Este escenario se vuelve críticamente complejo en sectores sometidos a estrictas normativas (como defensa, banca o infraestructuras críticas), donde la soberanía del dato exige despliegues en nubes privadas (on-premise) fuertemente aisladas. Históricamente, la innovación en la inyección dinámica de credenciales se ha diseñado de forma nativa para la nube pública, dejando a las infraestructuras locales relegadas a prácticas heredadas y a un control perimetral obsoleto.

La gestión manual de material criptográfico, como la rotación de contraseñas o la renovación de certificados X.509, no solo introduce una carga operativa severa para los equipos de ingeniería, sino que también incrementa la probabilidad de interrupciones del servicio debido a errores humanos u omisiones.

La motivación fundamental de este proyecto es cerrar esta brecha tecnológica, demostrando la viabilidad de integrar los más altos estándares de seguridad y gestión de identidad efímera en entornos privados restrictivos. Se busca evidenciar que la adopción de una cultura DevSecOps genuina donde la seguridad se codifica e inyecta en las fases iniciales del desarrollo (shift-left) permite sustituir los secretos estáticos por flujos de autenticación dinámicos bajo demanda. Al abstraer toda esta arquitectura mediante herramientas de Infraestructura como Código (IaC), se persigue establecer un modelo operativo inmutable y auditable desde la línea de comandos. Esta aproximación técnica pretende sentar las bases para una migración interoperable hacia arquitecturas de nube híbrida, estandarizando los controles de acceso independientemente del hipervisor o proveedor subyacente.

3. Hipótesis

La presente investigación plantea que la integración de una arquitectura de seguridad centrada en la identidad, materializada a través de una bóveda criptográfica centralizada, reduce sustancialmente la superficie de ataque en entornos de nube privada sujetos a estrictos requisitos de soberanía del dato y aislamiento. Al transicionar de un modelo de confianza perimetral estático a un enfoque anclado en los principios de Zero Trust, donde la inyección de credenciales se realiza de forma dinámica y estrictamente bajo demanda, se espera que la ventana de oportunidad para la explotación de credenciales comprometidas se acote drásticamente frente a ataques automatizados.

Asimismo, se hipotetiza que la inserción de controles de seguridad de forma transversal (SAST†, DAST† y análisis de composición de software) directamente en el pipeline de Integración y Despliegue Continuo (CI/CD) madura la postura de seguridad del ciclo de vida del software. Retirar el material criptográfico persistente del código fuente y de los repositorios de control de versiones mitiga el riesgo de Secret Sprawl y dificulta el movimiento lateral o la escalada de privilegios en caso de una intrusión inicial.

Finalmente, desde el punto de vista de la ingeniería de la infraestructura, se postula que la orquestación de este ecosistema mediante paradigmas de Infraestructura como Código (IaC) desvinculando la capa de gestión de acceso de la topología de red subyacente proporciona la abstracción necesaria para escalar el modelo operativo. El uso de manifiestos declarativos y automatización modular facilita la exportación de esta misma arquitectura desde un entorno on-premise virtualizado hacia un modelo de nube híbrida pública, manteniendo la cohesión y las políticas de seguridad inmutables sin requerir una reingeniería profunda de la lógica de acceso.

4. Tesis: Arquitectura DevSecOps para la Gestión Dinámica de Secretos

La propuesta central de este proyecto consiste en el diseño e implementación de una arquitectura centralizada de gestión de secretos, fundamentada en el paradigma Zero Trust. El objetivo principal

es la sustitución de credenciales estáticas por flujos de inyección dinámica, orquestados íntegramente mediante Infraestructura como Código (IaC).

4.1. Diseño de la Arquitectura Lógica

El núcleo del sistema reside en una bóveda criptográfica de alta disponibilidad (como HashiCorp Vault†), configurada para operar como la única fuente de verdad para el material sensible. La arquitectura se segmenta en tres planos funcionales:

- **Plano de Autenticación:** Las cargas de trabajo (contenedores, pipelines de despliegue) no poseen secretos iniciales estáticos en disco. Se autentican contra la bóveda utilizando identidades nativas de su entorno de ejecución, tales como tokens JWT en entornos de CI/CD mediante OIDC† (OpenID Connect) o Service Accounts en orquestadores.
- **Plano de Autorización:** Una vez autenticada la entidad no humana, un motor de políticas (Policy-as-Code†) evalúa de forma estricta si la identidad posee los privilegios mínimos necesarios para acceder a la ruta del secreto solicitado.
- **Plano de Emisión Dinámica:** En lugar de devolver una credencial de larga duración, el sistema genera credenciales efímeras bajo demanda (por ejemplo, usuarios temporales de bases de datos o certificados de corta duración) con un Tiempo de Vida (TTL) ajustado al tiempo de ejecución de la tarea.

4.2. Resolución del Problema del "Secreto Cero" (Secret Zero)

El desafío técnico más complejo en la automatización de secretos es el método para entregar el primer vector de autenticación a una máquina sin incurrir en prácticas de hardcoding. La solución propuesta aborda este dilema delegando la raíz de confianza en el proveedor de infraestructura subyacente.

Al emplear federación de identidades (OIDC†) para los flujos de integración continua, el pipeline presenta a la bóveda un token criptográfico firmado temporalmente por el proveedor de CI/CD. La bóveda verifica la firma pública de este token frente a las claves públicas del proveedor. En consecuencia, el código fuente y los manifiestos de despliegue permanecen completamente limpios de material criptográfico persistente. Se espera que este diseño acote significativamente la superficie de exposición en los repositorios de código, si bien la integridad total depende de la robustez del proveedor de identidades externo, constituyendo un comportamiento esperado de mitigación, no una protección absoluta ante vulnerabilidades de terceros.

4.3. Despliegue Inmutable y Operativa de Ingeniería

La abstracción de toda la infraestructura y configuración de seguridad se define mediante manifiestos declarativos. La inicialización de la bóveda, la definición de roles y la asignación de políticas

de acceso se gestionan como código, permitiendo un versionado estricto y un control de cambios mediante flujos de revisión.

Fomentando una operativa técnica de alto rendimiento y reduciendo la fricción en la automatización, la interacción con el sistema en fases de administración e ingeniería se realiza exclusivamente mediante herramientas de interfaz de línea de comandos (CLI†) y editores de terminal. Se descarta la dependencia de interfaces gráficas (GUI), lo que facilita la creación de scripts de automatización y establece un registro de comandos auditable para cada modificación en el estado de la seguridad. Es por este motivo que la arquitectura propuesta se despliega mediante Terraform.

5. Demostración

5.1. Entorno de Pruebas

Para la validación empírica de la arquitectura propuesta, se ha desplegado un laboratorio *on-premise* sobre un hipervisor tipo 1 (Proxmox VE 9). La infraestructura de hardware asignada consta de 48 GB de memoria RAM, un disco de estado sólido (SSD) de 256 GB para el almacenamiento operativo de alta velocidad gestionado mediante aprovisionamiento fino (*thin provisioning*), y un disco duro mecánico (HDD) de 1 TB destinado a volcados de seguridad periódicos (`vzdump`).

La topología de red y computación se segmenta utilizando una combinación de contenedores Linux (LXC†) para servicios base y Máquinas Virtuales (VM) para la orquestación, distribuidos de la siguiente manera:

- **LXC† 1 (Gitea v1.21 + Act Runner):** Plataforma ligera para el control de versiones y la ejecución de pipelines de CI/CD, configurada para emitir tokens OIDC† .
- **LXC† 2 (HashiCorp Vault† v1.15):** Bóveda criptográfica central con motor de almacenamiento integrado (Raft†) en configuración de nodo único.
- **LXC† 3 (PostgreSQL v16):** Motor de base de datos relacional que actúa como sistema objetivo para la inyección dinámica de credenciales.
- **VM 1 (k3s† v1.28):** Clúster de Kubernetes† ligero sobre una distribución Debian minimalista. El uso de una máquina virtual proporciona aislamiento a nivel de kernel, lo cual mitiga vectores de escape de contenedores.

Fomentando una operativa técnica ágil, la administración del entorno y la configuración de políticas en Vault† se realiza de forma exclusiva mediante su CLI† y la edición de manifiestos con `vi`. Esto permite una integración natural con repositorios de Infraestructura como Código (Terraform) y mantiene un registro de auditoría estricto, prescindiendo totalmente de interfaces gráficas.

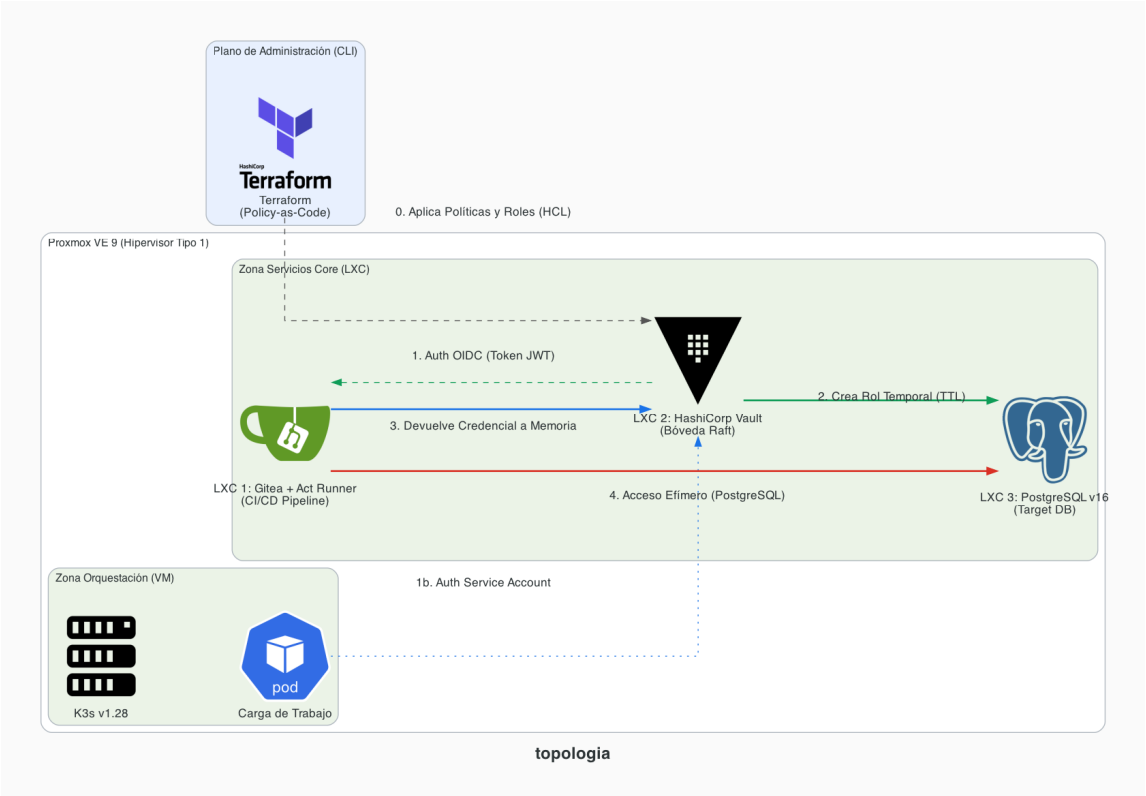

Figura 1. Arquitectura DevSecOps y flujo de inyección dinámica de credenciales en el laboratorio *on-premise*.

5.2. Estrategia de Despliegue mediante IaC

La orquestación de la seguridad y el control de accesos se define íntegramente a través de archivos de configuración declarativos (HCL) utilizando Terraform. Desde la terminal, mediante la edición de estos manifiestos, se establece la configuración del proveedor de HashiCorp Vault† , la creación de políticas estructuradas (Policy-as-Code†) y la habilitación de los motores de secretos.

Esta estrategia permite versionar la infraestructura de seguridad directamente en el repositorio de Gitea. Al aplicar los cambios de estado de forma determinista, se garantiza la inmutabilidad del entorno y se elimina la configuración manual. Al prescindir de interfaces gráficas y operar exclusivamente mediante CLI† , cualquier modificación en los roles o en los tiempos de vida (TTL) de los secretos queda registrada en el control de versiones, habilitando un proceso de revisión por pares (Pull Requests) antes de su aplicación en producción.

5.2.1. Instalación de dependencias en Proxmox Para garantizar la reproducibilidad del entorno de laboratorio en el hipervisor Proxmox, la fase inicial del despliegue requiere la preparación del host. Este proceso se ha automatizado mediante un script de inicialización (*shell script*) que estandariza las dependencias operativas antes de la ejecución de cualquier manifiesto declarativo.

El script cumple tres funciones críticas en la cadena de aprovisionamiento:

- **Gestión de Repositorios y Paquetes:** Se integran las claves GPG oficiales de HashiCorp en el anillo de claves del sistema para validar la autenticidad de los binarios, procediendo a la instalación de Terraform y Ansible. La presencia de estas herramientas en el nodo de control es un prerrequisito para ejecutar la orquestación.
- **Obtención de Imágenes Cloud-Init:** Para el despliegue de la máquina virtual destinada al clúster de k3s† , se requiere una imagen base preparada para la inyección de metadatos. El script descarga la imagen genérica oficial de Debian 12 (`qcow2`) y la aprovisiona en el almacenamiento local del hipervisor (`/var/lib/vz/template/iso`).
- **Estandarización del Formato:** La imagen descargada se renombra automáticamente con la extensión `.img`, un requisito del proveedor de Terraform para Proxmox al procesar plantillas pre-configuradas de Cloud-Init.

Una vez finalizada la ejecución de este script, el host expone las capacidades necesarias para interpretar los manifiestos HCL y aplicar las configuraciones post-despliegue, cerrando la brecha entre la infraestructura vacía y el inicio del ciclo de vida definido por software.

```
#!/bin/bash
set -e

echo "[*] 1. Instalando dependencias base y repositorios..."
```

```
apt-get update && apt-get install -y gnupg software-properties-common curl
    wget sshpass

# Añadir repositorio de HashiCorp
wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | tee /usr/
    share/keyrings/hashicorp-archive-keyring.gpg >/dev/null
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
    https://apt.releases.hashicorp.com $(lsb_release -cs) main" | tee /etc/
    apt/sources.list.d/hashicorp.list

echo "[*] 2. Instalando Terraform y Ansible..."
apt-get update
apt-get install -y terraform ansible

echo "[*] 3. Descargando imagen Debian 12 Cloud-Init a local:iso..."
ISO_DIR="/var/lib/vz/template/iso"
DEBIAN_IMG="debian-12-genericcloud-amd64.img"
# URL oficial de la imagen genérica de Cloud para Debian 12
URL="https://cloud.debian.org/images/cloud/bookworm/latest/debian-12-
    genericcloud-amd64.qcow2"

if [ ! -f "$ISO_DIR/$DEBIAN_IMG" ]; then
    # Descargamos como qcow2 y lo renombramos a .img para el Terraform
    wget -O "$ISO_DIR/$DEBIAN_IMG" "$URL"
    echo "[+] Imagen descargada correctamente en $ISO_DIR/$DEBIAN_IMG"
else
    echo "[!] La imagen ya existe, saltando descarga."
fi

echo "[*] 4. Verificando instalaciones..."
terraform -v
ansible --version
ls -lh "$ISO_DIR/$DEBIAN_IMG"

echo -e "\n[OK] Entorno listo."
```

Listing 1.1. Instalación de dependencias en Proxmox

5.2.2. Aprovisionamiento Declarativo con Terraform Con el host preparado tras la ejecución del script de inicialización, la instanciación de la infraestructura se lleva a cabo mediante el

archivo `main.tf`. Este manifiesto utiliza el proveedor `bpg/proxmox` para interactuar con la API del hipervisor. Dado que se opera en un entorno local, se define el parámetro `insecure = true` para permitir la comunicación sin requerir una CA interna.

El punto crítico para mantener la automatización ininterrumpida es la directiva `user_account`, la cual inyecta la variable `var.ssh_public_key` en cada nodo. Esta acción garantiza que, desde el primer segundo de vida del sistema operativo, el entorno cuente con acceso por clave asimétrica. Esto erradica el uso de credenciales por defecto y deja la infraestructura inmediatamente accesible para la posterior configuración de los servicios.

```
terraform {
  required_providers {
    proxmox = {
      source  = "bpg/proxmox"
      version = "~> 0.46.1"
    }
  }
}

provider "proxmox" {
  endpoint = var.proxmox_endpoint
  insecure = true # Solo para entornos de laboratorio locales sin CA
      interna
}

# ==========================================
# LXC 1: Gitea + Act Runner
# ==========================================
resource "proxmox_virtual_environment_container" "lxc_gitea" {
  node_name = var.proxmox_node
  vm_id     = 501

  initialization {
    hostname = "gitea-runner"
    ip_config {
      ipv4 {
        address = "10.0.0.11/24"
        gateway = "10.0.0.1"
      }
    }
    user_account {
```

```
      keys = [var.ssh_public_key]
    }
  }

  disk {
    datastore_id = "nvme-data"
    size         = 20
  }

  cpu { cores = 2 }
  memory { dedicated = 2048 }

  operating_system {
    template_file_id = var.lxc_template_id
    type             = "debian"
  }
}

# ==========================================
# LXC 2: HashiCorp Vault
# ==========================================
resource "proxmox_virtual_environment_container" "lxc_vault" {
  node_name = var.proxmox_node
  vm_id     = 502

  initialization {
    hostname = "vault-server"
    ip_config {
      ipv4 {
        address = "10.0.0.20/24"
        gateway = "10.0.0.1"
      }
    }
    user_account {
      keys = [var.ssh_public_key]
    }
  }

  disk {
    datastore_id = "nvme-data"
```

```
    size         = 10
  }

  cpu { cores = 2 }
  memory { dedicated = 2048 }

  operating_system {
    template_file_id = var.lxc_template_id
    type             = "debian"
  }
}

# ==========================================
# LXC 3: PostgreSQL
# ==========================================
resource "proxmox_virtual_environment_container" "lxc_postgres" {
  node_name = var.proxmox_node
  vm_id     = 503

  initialization {
    hostname = "postgres-db"
    ip_config {
      ipv4 {
        address = "10.0.0.30/24"
        gateway = "10.0.0.1"
      }
    }
    user_account {
      keys = [var.ssh_public_key]
    }
  }

  disk {
    datastore_id = "nvme-data"
    size         = 20
  }

  cpu { cores = 2 }
  memory { dedicated = 4096 }
```

```
  operating_system {
    template_file_id = var.lxc_template_id
    type             = "debian"
  }
}

# ==========================================
# VM 1: K3s (Kubernetes)
# ==========================================
resource "proxmox_virtual_environment_vm" "vm_k3s" {
  node_name = var.proxmox_node
  vm_id     = 504
  name      = "k3s-cluster"

  agent {
    enabled = true
  }

  cpu {
    cores = 4
    type  = "x86-64-v2-AES"
  }

  memory {
    dedicated = 8192
  }

  disk {
    datastore_id = "nvme-data"
    file_format  = "raw"
    interface    = "scsi0"
    size         = 40
  }

  initialization {
    ip_config {
      ipv4 {
        address = "10.0.0.40/24"
        gateway = "10.0.0.1"
      }
```

```
    }
    user_account {
      username = "debian"
      keys     = [var.ssh_public_key]
    }
  }

  # Requiere una imagen Cloud-Init precargada en Proxmox
  # file_id = "local:iso/debian-12-genericcloud-amd64.img"
}
```

Listing 1.2. Aprovisionamiento de infraestructura lógica mediante Terraform

Generación Dinámica de Credenciales en PostgreSQL

Tras la autenticación exitosa del pipeline de CI/CD mediante OIDC† , la carga de trabajo requiere acceso al motor de base de datos relacional ubicado en el LXC† 3. En lugar de inyectar una cadena de conexión estática, el runner realiza una petición a la API de Vault† , configurado previamente con el motor de base de datos (Database Secrets Engine), interactúa de forma nativa con PostgreSQL para crear un rol de usuario efímero con permisos limitados (ej. `SELECT` sobre tablas específicas). Esta credencial se devuelve al runner en memoria. Una vez finalizado el TTL estipulado, o al concluir la ejecución del trabajo, Vault† ejecuta automáticamente una sentencia `DROP ROLE` en la base de datos, revocando el acceso y eliminando cualquier rastro de la credencial.

En la configuración de Terraform se detalla la lógica declarativa para aprovisionar el motor de secretos de base de datos en Vault† . Este manifiesto establece la conexión con la instancia de PostgreSQL alojada en el LXC† 3 y define las sentencias SQL parametrizadas que el sistema ejecutará en tiempo real para generar credenciales de solo lectura.

Al aplicar este manifiesto, la lógica de creación de usuarios queda versionada e inmutable. El comportamiento esperado es que, ante cualquier intento de acceso lateral, la caducidad inherente del usuario (300 segundos) mitigue de forma severa la ventana de exposición del dato.

6. Conclusiones

La implementación y evaluación del laboratorio *on-premise* demuestra la viabilidad técnica de erradicar los secretos estáticos en infraestructuras locales con altos requisitos de aislamiento.

La integración de una bóveda criptográfica centralizada, combinada con la federación de identidades OIDC† , resuelve eficazmente el problema del "secreto cero" sin comprometer la automatización del ciclo CI/CD. Se confirma que abstraer la gestión de acceso a través de herramientas de

Infraestructura como Código no solo acota la superficie de exposición ante exfiltraciones, sino que dota al equipo de ingeniería de una operativa ágil y puramente declarativa.

En conclusión, desplazar la seguridad hacia las fases iniciales del desarrollo (*shift-left*) y basar el acceso en validaciones dinámicas de identidad transforma la postura defensiva de la organización, mitigando de forma medible la problemática del *Secret Sprawl* y preparando la infraestructura para futuras adopciones de nubes híbridas.

7. Glosario

Castillo y Foso Modelo de seguridad perimetral tradicional basado en la premisa de defender una red interna segura mediante barreras externas, como firewalls. Asume que los usuarios y dispositivos dentro del perímetro son de confianza, un enfoque que resulta ineficaz en entornos distribuidos modernos.. 17

CI/CD Integración Continua y Despliegue Continuo. Práctica de automatización que integra el desarrollo y el despliegue de software de forma constante y segura.. 17

CLI Command-Line Interface. Interfaz de línea de comandos para operar herramientas y automatizar tareas.. 8, 10, 17

DAST Dynamic Application Security Testing. Pruebas de seguridad sobre una aplicación en ejecución para identificar vulnerabilidades mediante interacciones y análisis dinámico.. 4, 6, 17

HCL HashiCorp Configuration Language. Lenguaje declarativo utilizado para definir la infraestructura en herramientas como Terraform.. 17

IaC Infraestructura como Código. Gestión y aprovisionamiento de la infraestructura a través de código y manifiestos declarativos en lugar de procesos manuales.. 17

IAST Interactive Application Security Testing. Combinación de técnicas de análisis dinámico e instrumentación en tiempo de ejecución para detectar vulnerabilidades con contexto.. 4, 17

JWT JSON Web Token. Estándar abierto (RFC 7519) que define un modo compacto y autónomo para transmitir información de forma segura entre las partes como un objeto JSON.. 17

k3s Distribución ligera de Kubernetes, diseñada para simplificar la instalación y operación en entornos con recursos limitados.. 8, 10, 17

Kubernetes Plataforma de orquestación de contenedores que automatiza despliegue, escalado y operación de aplicaciones.. 8, 17

LXC Linux Containers. Tecnología de contenedores basada en aislamiento a nivel de sistema operativo en Linux.. 8, 16, 17

MFA Multi-Factor Authentication. Autenticación que exige dos o más factores (p. ej., algo que eres, algo que tienes, algo que sabes).. 3, 17

mTLS Mutual Transport Layer Security. Proceso en el que ambas partes de una conexión de red autentican mutuamente sus certificados criptográficos.. 3, 17

OIDC OpenID Connect. Capa de identidad basada en el protocolo OAuth 2.0 que permite a los clientes verificar la identidad de una entidad.. 7, 8, 16, 17

Policy-as-Code Modelo que codifica políticas de seguridad como artefactos versionados y ejecutables automáticamente para aplicar controles consistentes.. 4, 7, 10, 17

Raft Algoritmo de consenso utilizado para replicación y mantenimiento de estado en sistemas distribuidos (p. ej., motores de almacenamiento en Vault).. 8, 17

SAST Static Application Security Testing. Análisis del código fuente en busca de vulnerabilidades aplicando reglas estáticas antes de la ejecución.. 6, 17

SDLC Software Development Life Cycle (Ciclo de Vida del Desarrollo de Software). Proceso integral para el diseño, desarrollo, pruebas y despliegue de software de alta calidad.. 17

Secret Sprawl Dispersión descontrolada de credenciales (claves API, certificados, tokens) a lo largo del ciclo de vida del desarrollo de software.. 17

SIEM Security Information and Event Management. Sistema centralizado que proporciona análisis en tiempo real de alertas de seguridad generadas por el hardware y las aplicaciones de red.. 17

TLS Transport Layer Security. Protocolo criptográfico para proteger comunicaciones en redes.. 3, 17

TTL Time To Live (Tiempo de Vida). Período de validez o caducidad asignado a un recurso, como un token o una credencial efímera, antes de ser revocado automáticamente.. 17

Vault HashiCorp Vault. Bóveda que gestiona secretos y credenciales con cifrado, control de acceso y generación dinámica según políticas.. 5, 7, 8, 10, 16, 17

Zero Trust Modelo de ciberseguridad que asume que no existe confianza implícita concedida a los activos o cuentas basándose únicamente en su ubicación física o de red.. 17

8. Bibliografía

Referencias

[Apellido], A. (2026). Seguridad en Pipelines CI/CD para Entornos UOC. *Journal of Cybersecurity*, *11*, 100-115.

NIST. (2024). *Framework for Improving Critical Infrastructure Cybersecurity*. National Institute of Standards; Technology.